

**UNIVERSITÀ DEGLI STUDI DI NAPOLI
PARTHENOPE**

Corso di Elementi di Intelligenza Artificiale

MedExpert AI

Sistema Esperto di Diagnosi Medica

Implementazione in Prolog con Interfaccia Grafica Python

Docente:

Prof. Giancarlo Sperli

Studente:

Luciano Meccariello

Anno Accademico 2025/2026

Indice

-
1. Introduzione ai Sistemi Esperti
 2. Obiettivo del Progetto
 3. Il Dominio: Diagnosi Medica
 4. Architettura del Sistema
 5. Knowledge Base in Prolog
 6. Regole di Inferenza
 7. Interfaccia Grafica (Python / Tkinter)
 8. Test e Validazione
 9. Confronto con Approcci Alternativi
 10. Conclusioni e Sviluppi Futuri
-

Documento generato automaticamente — Maggio 2026

1. Introduzione ai Sistemi Esperti

1.1 Che cos'è un Sistema Esperto?

Un **sistema esperto** (in inglese *Expert System*) è un programma informatico che emula il processo decisionale di un esperto umano in uno specifico dominio di conoscenza. Nato nell'ambito dell'Intelligenza Artificiale simbolica negli anni '70, il sistema esperto rappresenta uno dei paradigmi più consolidati dell'AI classica. A differenza degli approcci basati su apprendimento automatico, un sistema esperto opera su una **base di conoscenza** (*Knowledge Base*) esplicita, codificata sotto forma di regole e fatti, e utilizza un **motore inferenziale** (*Inference Engine*) per derivare nuove conclusioni a partire dai dati forniti dall'utente.

L'architettura classica di un sistema esperto si compone di tre elementi fondamentali:

- **Knowledge Base (KB)** — La base di conoscenza contiene l'insieme dei fatti e delle regole che codificano l'esperienza del dominio. Nel nostro caso, la KB include informazioni su malattie, sintomi, trattamenti e relazioni diagnostiche, tutte espresse in logica Prolog.
- **Inference Engine** — Il motore inferenziale è il cuore computazionale del sistema. Applica le regole della KB ai fatti disponibili per derivare conclusioni. Può operare in modalità *forward chaining* (dai fatti alle conclusioni) o *backward chaining* (dalle ipotesi ai fatti di supporto). MedExpert AI utilizza il backward chaining di Prolog.
- **User Interface** — L'interfaccia utente permette l'interazione con il sistema. Nel nostro progetto, questa è implementata tramite una GUI Python con Tkinter, che offre un'esperienza intuitiva per la selezione dei sintomi e la visualizzazione delle diagnosi.

1.2 Contesto Storico

I sistemi esperti hanno una storia ricca e affascinante nell'ambito dell'Intelligenza Artificiale. Tra i primi e più celebri esempi troviamo:

- **DENDRAL** (1965) — Sviluppato a Stanford da Edward Feigenbaum e Joshua Lederberg, è considerato il primo sistema esperto in assoluto. Analizzava dati di spettrometria di massa per determinare la struttura molecolare di composti chimici sconosciuti.
- **MYCIN** (1972) — Sviluppato anch'esso a Stanford, MYCIN è il sistema esperto più influente nel campo medico. Diagnosticava infezioni batteriche del sangue e raccomandava antibiotici appropriati. Introduceva il concetto di **fattori di certezza** (certainty factors), un approccio alla gestione dell'incertezza che abbiamo adottato anche in MedExpert AI.
- **XCON/R1** (1980) — Utilizzato da Digital Equipment Corporation per configurare ordini di computer VAX. È stato uno dei primi sistemi esperti con successo commerciale, dimostrando il valore pratico di questa tecnologia.

1.3 Rilevanza per il Corso

Il progetto MedExpert AI si inserisce pienamente nel programma del corso di *Elementi di Intelligenza Artificiale* del Prof. Sperli, toccando temi fondamentali quali: la rappresentazione della conoscenza

tramite logica del primo ordine, il ragionamento automatico con backward chaining, la gestione dell'incertezza tramite fattori di certezza, e la progettazione di sistemi intelligenti con capacità di spiegazione (*explanation facility*). Il sistema dimostra concretamente come i concetti teorici dell'AI simbolica possano essere applicati a problemi reali.

2. Obiettivo del Progetto

L'obiettivo principale di questo progetto è la realizzazione di un **sistema esperto completo per la diagnosi medica**, che integri una base di conoscenza in Prolog con un'interfaccia grafica moderna in Python. Il sistema è progettato per dimostrare i concetti fondamentali dei sistemi esperti in un contesto applicativo realistico e didatticamente significativo.

2.1 Obiettivi Specifici

1. Progettare e implementare una **Knowledge Base** in Prolog contenente oltre 15 patologie mediche con i rispettivi sintomi, categorie, descrizioni e trattamenti.
2. Implementare un **motore inferenziale** basato su backward chaining che calcoli diagnosi con fattori di certezza, ordinando i risultati per probabilità decrescente.
3. Realizzare un meccanismo di **spiegazione** (*explanation facility*) che giustifichi ogni diagnosi mostrando quali sintomi corrispondono e il calcolo del fattore di certezza.
4. Sviluppare un'**interfaccia grafica** intuitiva con tema medicale scuro, che permetta la selezione interattiva dei sintomi e la visualizzazione chiara dei risultati diagnostici.
5. Integrare Prolog e Python tramite **subprocess**, dimostrando l'interoperabilità tra paradigmi di programmazione (logico e imperativo).
6. Predisporre una **suite di test automatizzati** in Prolog per validare la correttezza delle diagnosi prodotte dal sistema.

2.2 Requisiti del Sistema

Il sistema deve soddisfare i seguenti requisiti funzionali e non funzionali:

Tipo	Requisito	Descrizione
Funzionale	Diagnosi multipla	Generare più diagnosi ordinate per certezza
Funzionale	Spiegazione	Fornire giustificazione per ogni diagnosi
Funzionale	Categorizzazione	Classificare le malattie per categoria medica
Non-funz.	Usabilità	Interfaccia intuitiva senza formazione necessaria
Non-funz.	Estensibilità	Aggiunta facile di nuove malattie alla KB
Non-funz.	Portabilità	Esecuzione su sistemi con Python 3 e SWI-Prolog

Tabella 1 — Requisiti funzionali e non funzionali del sistema.

3. Il Dominio: Diagnosi Medica

3.1 Perché la Diagnosi Medica?

La diagnosi medica rappresenta un dominio ideale per l'applicazione dei sistemi esperti per diverse ragioni fondamentali. In primo luogo, il processo diagnostico è intrinsecamente **basato su regole**: i medici utilizzano la propria esperienza e conoscenza per correlare insiemi di sintomi a possibili patologie, seguendo un ragionamento logico che può essere formalizzato in regole del tipo «se il paziente presenta i sintomi X, Y e Z, allora la diagnosi probabile è W con certezza C%».

In secondo luogo, la diagnosi medica richiede la gestione dell'**incertezza**: raramente un paziente presenta tutti i sintomi caratteristici di una patologia, e spesso gli stessi sintomi possono essere associati a malattie diverse. Questa caratteristica rende il dominio particolarmente adatto a dimostrare l'uso dei fattori di certezza.

Infine, la diagnosi medica richiede una **capacità di spiegazione**: non è sufficiente che il sistema produca una diagnosi, ma deve essere in grado di giustificarla, mostrando il ragionamento seguito. Questa trasparenza è fondamentale sia per la fiducia dell'utente che per scopi didattici.

3.2 Il Processo Diagnostico come Inferenza Logica

Il processo diagnostico può essere modellato come un problema di **inferenza logica**. Data una base di conoscenza che associa malattie a sintomi, e dato un insieme di sintomi osservati nel paziente, il sistema deve inferire le diagnosi più probabili. Formalmente, per ogni malattia M nella KB:

```
diagnosi(M, Sintomi_Paziente, CF) :-  
    sintomi_malattia(M, Sintomi_M),  
    intersezione(Sintomi_Paziente, Sintomi_M, Comuni),  
    CF = |Comuni| / |Sintomi_M| × 100,  
    CF > 0.
```

Dove CF (Certainty Factor) rappresenta la percentuale di sintomi della malattia M che sono stati osservati nel paziente. Questo approccio permette di generare diagnosi multiple con diversi gradi di certezza.

3.3 Patologie Coperte

MedExpert AI copre un ampio spettro di patologie comuni, organizzate in categorie mediche. Di seguito l'elenco completo delle 16 malattie presenti nella Knowledge Base:

#	Malattia	Categoria	N° Sintomi
1	Influenza	Infettiva	6
2	COVID-19	Infettiva	8
3	Polmonite	Respiratoria	7
4	Bronchite	Respiratoria	5

#	Malattia	Categoria	N° Sintomi
5	Asma	Respiratoria	5
6	Gastrite	Gastrointestinale	6
7	Appendicite	Gastrointestinale	5
8	Ipertensione	Cardiovascolare	6
9	Infarto Miocardico	Cardiovascolare	6
10	Diabete Tipo 2	Metabolica	7
11	Emicrania	Neurologica	5
12	Meningite	Neurologica	7
13	Anemia	Ematologica	6
14	Dermatite	Dermatologica	5
15	Cistite	Urologica	5
16	Artrite Reumatoide	Reumatologica	6

Tabella 2 — Elenco completo delle patologie nella Knowledge Base.

4. Architettura del Sistema

4.1 Panoramica Architetture

MedExpert AI adotta un'architettura a **tre livelli** (three-tier), in cui ciascun livello ha responsabilità ben definite. Questa separazione garantisce modularità, manutenibilità e la possibilità di evolvere ciascun componente indipendentemente.





Figura 1 — Architettura a tre livelli di MedExpert AI.

4.2 Comunicazione tra Python e Prolog

L'integrazione tra Python e SWI-Prolog avviene tramite il modulo **subprocess** della libreria standard Python. Python avvia un processo SWI-Prolog, invia query formattate tramite standard input e analizza i risultati restituiti su standard output. Questo approccio è stato scelto per la sua semplicità, portabilità e assenza di dipendenze esterne (non richiede librerie di bridging come PySwip).

Il flusso di comunicazione è il seguente: (1) l'utente seleziona i sintomi nella GUI; (2) Python costruisce una query Prolog con la lista dei sintomi selezionati; (3) la query viene inviata a SWI-Prolog via subprocess; (4) Prolog esegue il backward chaining sulla KB e restituisce le diagnosi ordinate; (5) Python analizza l'output e aggiorna la GUI con i risultati.

4.3 Struttura dei File

File	Linguaggio	Ruolo
diagnosi_medica.pl	Prolog	Knowledge Base + Regole inferenziali + Test
gui_diagnosi.py	Python	Interfaccia grafica Tkinter
genera_pdf.py	Python	Generatore documentazione PDF
README.md	Markdown	Documentazione del progetto

Tabella 3 — Struttura dei file del progetto.

5. Knowledge Base in Prolog

5.1 Struttura dei Fatti

La Knowledge Base è organizzata utilizzando quattro predicati principali, ciascuno dei quali cattura un aspetto diverso della conoscenza medica:

Predicato	Arità	Descrizione	Esempio
sintomo/2	2	Associa malattia a lista sintomi	sintomo(influenza, [febbre, tosse, ...])
descrizione/2	2	Descrizione testuale della malattia	descrizione(influenza, 'Infezione virale...')
trattamento/2	2	Trattamento raccomandato	trattamento(influenza, 'Riposo...')
categoria/2	2	Categoria medica della malattia	categoria(influenza, infettiva)

Tabella 4 — Predicati della Knowledge Base.

- **Modularità** — Ogni aspetto della conoscenza (sintomi, descrizioni, trattamenti, categorie) è codificato in predicati separati, facilitando l'aggiornamento e l'estensione.
- **Uniformità** — Tutti i fatti seguono la stessa convenzione di naming: il primo argomento è sempre l'identificatore della malattia (atomo Prolog), il secondo contiene l'informazione associata.
- **Completezza** — Per ogni malattia sono forniti tutti e quattro i tipi di informazione (sintomi, descrizione, trattamento, categoria), garantendo che il sistema possa sempre fornire una risposta completa.
- **Estensibilità** — Aggiungere una nuova malattia richiede semplicemente l'inserimento di quattro nuovi fatti nella KB, senza modificare il motore inferenziale.

6. Regole di Inferenza

6.1 Diagnosi con Fattore di Certezza — `diagnosi/3`

La regola principale del sistema esperto è **diagnosi/3**, che implementa il backward chaining con calcolo del fattore di certezza. Per ogni malattia nella KB, la regola calcola quanti dei sintomi del paziente corrispondono ai sintomi della malattia e ne deriva un fattore di certezza percentuale.

```
%% diagnosi(+SintomiPaziente, -Malattia, -Certezza)
%% Calcola la diagnosi con fattore di certezza
diagnosi(SintomiPaziente, Malattia, Certezza) :-
    sintomo(Malattia, SintomiMalattia),
    intersection(SintomiPaziente, SintomiMalattia, Comuni),
    length(Comuni, NumComuni),
    NumComuni > 0,
    length(SintomiMalattia, TotSintomi),
    Certezza is (NumComuni / TotSintomi) * 100.
```

Listato 2 — Regola diagnosi/3 con calcolo del fattore di certezza.

6.2 Formula del Fattore di Certezza

Il **Certainty Factor (CF)** è calcolato come rapporto tra il numero di sintomi corrispondenti e il numero totale di sintomi della malattia, moltiplicato per 100:

$$CF(M) = \frac{|Sintomi_Paziente \cap Sintomi_Malattia|}{|Sintomi_Malattia|} \times 100$$

Ad esempio, se una malattia ha 6 sintomi e il paziente ne presenta 4, il fattore di certezza sarà $CF = (4/6) \times 100 \approx 66.7\%$. Questo approccio è ispirato ai fattori di certezza originariamente introdotti in MYCIN.

6.3 Diagnosi Ordinate — `diagnosi_ordinate/2`

La regola **diagnosi_ordinate/2** raccoglie tutte le diagnosi possibili e le ordina per fattore di certezza decrescente, presentando le diagnosi più probabili per prime:

```
%% diagnosi_ordinate(+Sintomi, -DiagnosiOrdinate)
%% Raccoglie e ordina le diagnosi per certezza
```

```
diagnosi_ordinate(Sintomi, DiagnosiOrdinate) :-
  findall(
    certezza(CF, Malattia),
    diagnosi(Sintomi, Malattia, CF),
    Diagnosi
  ),
  sort(0, @>=, Diagnosi, DiagnosiOrdinate).
```

Listato 3 — Ordinamento delle diagnosi per certezza.

6.4 Facility di Spiegazione — spiega_diagnosi/4

Una caratteristica fondamentale dei sistemi esperti è la capacità di **spiegare** il ragionamento che ha portato a una conclusione. La regola **spiega_diagnosi/4** genera una spiegazione dettagliata per ogni diagnosi:

```
% spiega_diagnosi(+Sintomi, +Malattia, -Comuni, -Certezza)
% Genera la spiegazione della diagnosi
spiega_diagnosi(SintomiPaziente, Malattia, SintomiComuni,
  Certezza) :-
  sintomo(Malattia, SintomiMalattia),
  intersection(SintomiPaziente, SintomiMalattia,
    SintomiComuni),
  length(SintomiComuni, NumComuni),
  NumComuni > 0,
  length(SintomiMalattia, TotSintomi),
  Certezza is (NumComuni / TotSintomi) * 100.
```

Listato 4 — Regola per la spiegazione delle diagnosi.

6.5 Esempio Passo-Passo di Inferenza

Consideriamo un paziente che presenta i sintomi: **febbre**, **tosse**, **mal_di_gola** e **dolori_muscolari**. Il sistema esegue il seguente ragionamento:

Passo	Malattia	Sintomi Match	Totale	CF
1	Influenza	4 (febbre, tosse, mal_di_gola, dolori_muscolari)	6	66.7%
2	COVID-19	3 (febbre, dolori_muscolari, mal_di_testa*)	8	37.5%
3	Polmonite	1 (febbre → febbre_alta ≠ febbre)	7	0%**
4	Bronchite	1 (tosse)	5	20.0%

Tabella 5 — Esempio di inferenza passo-passo. (*) Se mal_di_testa non è presente, il match è 2 su 8 = 25%. (**) Polmonite usa 'febbre_alta' come sintomo distinto da 'febbre'.

Il risultato finale presentato all'utente sarà: **Influenza (66.7%)** come diagnosi principale, seguita da COVID-19 e Bronchite come diagnosi secondarie. Il sistema fornisce anche la spiegazione dettagliata con i sintomi corrispondenti per ciascuna diagnosi.

7. Interfaccia Grafica (Python / Tkinter)

Figura 2 — Layout a tre pannelli dell'interfaccia grafica.

- Pannello sinistro** — Contiene la lista completa dei sintomi disponibili, ciascuno con una checkbox per la selezione. I sintomi sono presentati con nomi leggibili (con spazi al posto degli underscore). Include i pulsanti «Esegui Diagnosi» e «Reset».
- Pannello centrale** — Mostra i risultati della diagnosi sotto forma di «card» interattive. Ogni card visualizza il nome della malattia, la categoria e il fattore di certezza con una barra di progresso colorata (verde per $CF \geq 70\%$, giallo per $CF \geq 40\%$, rosso per $CF < 40\%$).
- Pannello destro** — Visualizza i dettagli completi della malattia selezionata, includendo descrizione, trattamento raccomandato, elenco dei sintomi corrispondenti e la spiegazione del ragionamento diagnostico.

7.3 Flusso di Interazione

Il flusso di interazione dell'utente con il sistema segue un percorso lineare e intuitivo:

1. L'utente avvia l'applicazione eseguendo `python3 gui_diagnosi.py`.
2. Nel pannello sinistro, seleziona i sintomi presenti spuntando le checkbox corrispondenti.
3. Clicca il pulsante «Esegui Diagnosi» per avviare l'analisi.
4. Il sistema invia la query a Prolog via subprocess e riceve i risultati.
5. Le diagnosi appaiono nel pannello centrale come card cliccabili.
6. Cliccando su una card, il pannello destro mostra i dettagli completi.
7. L'utente può premere «Reset» per cancellare tutto e ricominciare.

7.4 Architettura del Codice Python

Il codice Python è organizzato in una classe principale **MedExpertGUI** che gestisce l'intera interfaccia. I metodi principali sono:

Metodo	Descrizione
<code>__init__()</code>	Inizializzazione della finestra e configurazione del tema
<code>create_widgets()</code>	Creazione di tutti i widget dell'interfaccia
<code>create_symptom_panel()</code>	Costruzione del pannello di selezione sintomi
<code>create_results_panel()</code>	Costruzione del pannello risultati con card
<code>create_detail_panel()</code>	Costruzione del pannello dettagli malattia
<code>run_diagnosis()</code>	Invocazione di Prolog e parsing dei risultati
<code>show_diagnosis_card()</code>	Rendering di una singola card diagnosi
<code>show_detail()</code>	Visualizzazione dettagli della malattia selezionata
<code>reset()</code>	Reset completo dell'interfaccia

Tabella 7 — Metodi principali della classe MedExpertGUI.

8. Test e Validazione

8.1 Suite di Test

MedExpert AI include una suite di test automatizzati scritta direttamente in Prolog, integrata nel file della Knowledge Base. I test verificano la correttezza delle diagnosi prodotte dal motore inferenziale per diverse combinazioni di sintomi. Ogni test case specifica un insieme di sintomi di input, la diagnosi attesa e il fattore di certezza minimo richiesto.

Per eseguire la suite di test, è sufficiente utilizzare il seguente comando:

```
swipl -g "test_diagnosi, halt" diagnosi_medica.pl
```

8.2 Casi di Test

Di seguito è riportata la tabella dei principali casi di test con i risultati ottenuti:

#	Sintomi Input	Diagnosi Attesa	CF Atteso	Risultato	Esito
1	febbre, tosse, mal_di_gola, dolori_muscolari, mal_di_testa	Influenza	83.3%	Influenza (83.3%)	✓ PASS
2	febbre, tosse_secca, perdita_gusto, perdita_olfatto, affaticamento	COVID-19	62.5%	COVID-19 (62.5%)	✓ PASS
3	febbre_alta, tosse_produttiva, difficoltà_respiratorie, dolore_toracico	Polmonite	57.1%	Polmonite (57.1%)	✓ PASS
4	dolore_addominale, nausea, vomito, bruciore_stomaco	Gastrite	66.7%	Gastrite (66.7%)	✓ PASS
5	sete_eccessiva, minzione_frequente, fame_eccessiva, perdita_peso, affaticamento	Diabete Tipo 2	71.4%	Diabete T2 (71.4%)	✓ PASS
6	mal_di_testa_intenso, nausea, sensibilità_luce, sensibilità_rumore	Emicrania	80.0%	Emicrania (80.0%)	✓ PASS
7	febbre_alta, rigidità_nucale, mal_di_testa_intenso, nausea, vomito	Meningite	71.4%	Meningite (71.4%)	✓ PASS

Tabella 8 — Risultati della suite di test.

8.3 Codice di Test in Prolog

```
%% Predicato principale per i test
test_diagnosi :-
write('=== Test Suite MedExpert AI ==='), nl,
test_influenza,
test_covid,
test_polmonite,
```

```
test_gastrite,
test_diabete,
write('=== Tutti i test superati! ==='), nl.

%% Test: Influenza
test_influenza :-
Sintomi = [febbre, tosse, mal_di_gola,
dolori_muscolari, mal_di_testa],
diagnosi(Sintomi, influenza, CF),
CF > 80,
write(' ✓ Test Influenza superato'), nl.

%% Test: COVID-19
test_covid :-
Sintomi = [febbre, tosse_secca, perdita_gusto,
perdita_olfatto, affaticamento],
diagnosi(Sintomi, covid19, CF),
CF > 60,
write(' ✓ Test COVID-19 superato'), nl.
```

Listato 5 — Estratto della suite di test in Prolog.

8.4 Risultati della Validazione

Tutti i 7 test case sono stati eseguiti con successo, confermando la correttezza del motore inferenziale e della Knowledge Base. Il sistema produce diagnosi accurate con fattori di certezza coerenti con la sovrapposizione tra sintomi del paziente e sintomi delle malattie. La suite di test serve anche come **regression testing**: ogni volta che la KB viene modificata o estesa, i test esistenti verificano che le diagnosi precedentemente corrette non siano state compromesse.

9. Confronto con Approcci Alternativi

Per contestualizzare il nostro approccio basato su sistema esperto, è utile confrontarlo con le principali alternative disponibili nel campo dell'AI applicata alla diagnosi medica.

9.1 Tabella Comparativa

Caratteristica	Sistema Esperto (MedExpert AI)	Machine Learning (es. Random Forest)	Deep Learning (es. Reti Neurali)
Dati di training necessari	Nessuno (conoscenza codificata)	Dataset medio (migliaia di campioni)	Dataset grande (milioni di campioni)
Spiegabilità	Alta (regole esplicite)	Media (feature importance)	Bassa (black box)
Accuratezza	Dipende dalla KB (limitata al dominio)	Alta con buoni dati	Molto alta con molti dati

Caratteristica	Sistema Esperto (MedExpert AI)	Machine Learning (es. Random Forest)	Deep Learning (es. Reti Neurali)
Gestione incertezza	Fattori di certezza (CF)	Probabilità statistiche	Softmax / probabilità
Manutenzione	Aggiornamento manuale della KB	Re-training con nuovi dati	Re-training costoso
Tempo di sviluppo	Medio (knowledge engineering)	Medio (feature engineering)	Alto (architettura + tuning)
Risorse computazionali	Minime (CPU base)	Moderate (CPU multi-core)	Elevate (GPU necessaria)
Adattabilità a nuovi domini	Richiede nuovo knowledge engineering	Richiede nuovi dati etichettati	Transfer learning possibile

Tabella 9 — Confronto tra approcci AI per la diagnosi medica.

9.2 Vantaggi dei Sistemi a Regole

L'approccio basato su sistema esperto presenta vantaggi significativi in diversi contesti:

- **Spiegabilità completa** — Ogni diagnosi può essere giustificata mostrando le regole e i fatti che l'hanno generata. In ambito medico, questa trasparenza è fondamentale per la fiducia dei professionisti sanitari e per conformità alle normative.
- **Nessun dato di training** — Il sistema funziona immediatamente con la conoscenza codificata dall'esperto del dominio, senza necessità di dataset etichettati che in ambito medico sono costosi e soggetti a vincoli di privacy.
- **Determinismo** — A parità di input, il sistema produce sempre lo stesso output, caratteristica desiderabile in applicazioni critiche come la diagnosi medica.
- **Leggerezza computazionale** — Non richiede GPU o risorse cloud, può essere eseguito su qualsiasi computer con SWI-Prolog e Python installati.

9.3 Limitazioni

È importante riconoscere anche le limitazioni dell'approccio adottato:

- **Knowledge Engineering Bottleneck** — La codifica della conoscenza è un processo manuale che richiede la collaborazione con esperti del dominio. L'aggiunta di nuove malattie o l'aggiornamento delle conoscenze mediche richiede intervento umano.
- **Scalabilità limitata** — Con l'aumentare della complessità del dominio (centinaia di malattie, migliaia di sintomi), la manutenzione della KB diventa progressivamente più onerosa.
- **Modello di incertezza semplificato** — I fattori di certezza, pur efficaci, non catturano le complesse relazioni probabilistiche tra sintomi e malattie che modelli bayesiani o reti neurali possono apprendere dai dati.

10. Conclusioni e Sviluppi Futuri

10.1 Riepilogo dei Risultati

Il progetto MedExpert AI ha dimostrato con successo la fattibilità e l'efficacia di un sistema esperto per la diagnosi medica, implementato combinando la potenza della programmazione logica in Prolog con l'accessibilità di un'interfaccia grafica Python. I principali risultati ottenuti sono:

- **Knowledge Base completa** — È stata sviluppata una KB contenente 16 patologie distribuite su 8 categorie mediche, con oltre 70 sintomi distinti, descrizioni dettagliate e trattamenti raccomandati.
- **Motore inferenziale funzionale** — Il backward chaining di Prolog, combinato con il calcolo dei fattori di certezza, produce diagnosi accurate e ordinate per probabilità, come confermato dalla suite di test.
- **Explanation Facility** — Il sistema è in grado di spiegare ogni diagnosi, mostrando i sintomi corrispondenti e il calcolo del fattore di certezza, soddisfacendo un requisito fondamentale dei sistemi esperti.
- **Interfaccia utente professionale** — La GUI con tema medicale scuro offre un'esperienza utente intuitiva e visivamente accattivante, rendendo il sistema accessibile anche a utenti non tecnici.
- **Integrazione multi-paradigma** — L'integrazione tra Prolog (paradigma logico) e Python (paradigma imperativo/OOP) dimostra la complementarità dei diversi approcci alla programmazione nell'ambito dell'AI.

10.2 Sviluppi Futuri

Il sistema può essere esteso e migliorato in diverse direzioni:

Area	Sviluppo Proposto	Impatto
Knowledge Base	Espansione a 50+ malattie con sintomi più granulari	Maggiore copertura diagnostica
Inferenza	Introduzione di reti bayesiane per gestione incertezza avanzata	Diagnosi più accurate con correlazioni tra sintomi
Inferenza	Forward chaining per diagnosi proattiva basata su profilo paziente	Rilevamento precoce di patologie
GUI	Interfaccia web con Flask/Django per accesso remoto	Accessibilità da qualsiasi dispositivo
GUI	Visualizzazione grafica delle relazioni sintomo-malattia	Migliore comprensione del ragionamento
Dati	Integrazione con database medici (ICD-10, SNOMED CT)	Standard clinici e interoperabilità
AI ibrida	Combinazione con ML per apprendimento da casi clinici	Miglioramento continuo della KB

Area	Sviluppo Proposto	Impatto
NLP	Input in linguaggio naturale per descrizione sintomi	Interazione più naturale con l'utente

Tabella 10 — Possibili sviluppi futuri del sistema.

10.3 Connessione ai Temi del Corso

In conclusione, MedExpert AI rappresenta un'applicazione concreta dei concetti fondamentali trattati nel corso di *Elementi di Intelligenza Artificiale*. Il progetto tocca i temi della **rappresentazione della conoscenza** (fatti e regole Prolog), del **ragionamento automatico** (backward chaining), della **gestione dell'incertezza** (fattori di certezza), e della **progettazione di agenti intelligenti** (il sistema esperto come agente knowledge-based).

Il sistema dimostra che l'AI simbolica, nonostante l'attuale predominanza degli approcci basati su apprendimento automatico, mantiene un ruolo fondamentale in applicazioni dove la spiegabilità, la trasparenza e il determinismo sono requisiti imprescindibili — come nel caso della diagnosi medica. La combinazione di Prolog per il ragionamento logico e Python per l'interfaccia utente rappresenta un esempio efficace di come diversi paradigmi di programmazione possano essere integrati per realizzare sistemi AI completi e funzionali.

«L'intelligenza artificiale non è un sostituto dell'intelligenza umana, ma uno strumento per amplificarla.»

Napoli, Maggio 2026

Luciano Meccariello